

CosmicMart — Cosmo Technical Spec

Exact scripted copy, state machine, and matching logic behind the Cosmo shopping assistant — everything needed to rebuild it from scratch.

Architecture (4 layers)

- **UI panel** (`CosmoFullFlow.jsx`) — chat state machine, all copy
- **Orchestrator** (`cosmo.service.ts`) — intent parsing, then fires all four agents in parallel via `Promise.all`
- **Keyword maps** (`keywordMaps.ts`) — shared category / colour / occasion resolution used by both the parser and the gates
- **Four agents** — localization, trends, friend, recommendation — each returns a typed result keyed by `productId`

Greeting & Privacy Prompt

Cosmo's opening line, shown once at the start of every session:

“Hi, I'm Cosmo ♦ What are you shopping for today? I can help you by finding trending items, searching what your friends like, and recommending items based on previous purchasing history! Before we begin — you're browsing in private mode, let me know if you want to change that!”

Privacy quick replies: ["Keep browsing privately", "Change privacy settings"]

- “Keep browsing privately” → “Sounds good — staying in private mode ♦”
- “Change privacy settings” → “No problem — you can manage privacy settings anytime from your account.”

Intake Questions

Category, colour, and occasion are all parsed out of the customer's first free-text message — never asked separately. After that, exactly three questions are asked in this fixed order, one at a time, and nothing already known is ever re-asked:

Step	Prompt	Quick replies
budget	“What's your budget?”	Under \$50 / \$50–100 / \$100–150 / No limit
size	“What's your size?”	XS / S / M / L / XL
season	“Last one — what's the season?”	Spring / Summer / Fall / Winter

Transition line once all three are answered: “On it — running four agents in parallel now ♦”

Agent Run Animation

Header: “Running your search across 4 agents...” Each row shows a label and animates queued → running... → done:

- “Localization Agent — checking regional stock” (550ms)
- “Trend Agent — scoring seasonal fit” (700ms)
- “Friend Influence Agent — collecting room votes” (500ms)
- “Recommendation Agent — applying hard gates” (850ms)

The UI waits until `longestAgentMs + 150ms` before revealing results, so the progress bar never jumps ahead of the real `Promise.all` resolution.

Results

Intro bubble shown above the ranked list:

“Here's what passed the budget, category, and colour gates{ in size X} — ranked by fit, trend, rating, and your friends in room **{roomCode}**. Tap a card, then ask me to invite friends for advice.”

Empty-results state (the exact string behind the bug report that started this build):

“No items passed all three gates (budget, category, colour) for this search. Try widening your budget or dropping the colour preference.”

Each result card shows: emoji thumbnail, name, badges (★ Top pick / Selling fast if popularity ≥ 85 / Out of stock / Ships in Nd), price, a friend-votes line (“Liked by X, Y” or “n/m friends voted up”) plus trend score, star rating, an “Invite friends” button, and an Add to bag / Out of stock button.

Off-script reply if the customer types something during results that isn't an invite request:

“Happy to help with the {item} — add it to your bag, pick a different card above, or say “invite my friends” for advice 💎”

Friend Invite Flow

Trigger: free text contains “invite” AND (“friend” or “advice” or “call”) — or the customer clicks the per-card “Invite friends” button.

Cosmo: “Great choice on the {item} 💎 Who's around to weigh in?”

Friend picker header: “Who's active right now?” — checkboxes enabled only for active friends; offline ones are shown but disabled.

Roster (`friend.service.ts`):

- Active: Elizabeth, Mario, Maya, Priya, Leo
- Offline: Jordan, Sam, Aisha

On invite: “{Name} and {Name} joined the call 💎”, then each friend's line drops in staggered at 500ms + $i \times 700$ ms.

Scripted lines for the two named friends; everyone else gets a random pick (seeded by name so it's stable across re-renders) from a generic pool:

- Elizabeth → “I love that dress on you!”
- Mario → “I love it and I don't like a lot of things.”
- Generic pool: “This looks amazing on you!” / “Get it, no question.” / “I'd wear this in a heartbeat.” / “Yes! That's the one.”

After the last friend line: “Your friends love it — would you like to add it to cart?” with quick replies “Yes, add to cart” / “Not right now”.

Affirmative detection: text is exactly “yes”, starts with “yes”, or contains “add it” / “add to cart”.

- Confirmed → “Added to your bag ♦ Anything else I can help with?”
- Declined → “No problem — let me know if you'd like to add it later.”

Edge Cases (always-on, checked before anything else)

This intercept runs on **every** message, in every phase of the conversation — intake, results, even mid friend-chat — before any agent or state-machine logic touches the input.

1. Harmful input — triggers if the message contains any of: hate, don't like you, dont like you, shut up, stupid, idiot, useless, kill yourself, screw you.

“I'm a virtual shopping assistant. If you no longer wish to speak with me, I can refer you to a representative.”
+ phone card “☎ (123) 123-1234”, with a “Let's keep shopping” button that returns to the prior phase.

2. Out-of-catalog — triggers if the message mentions an item CosmicMart doesn't sell (headphones, laptop, phone, tv, gaming console, camera, monitor, keyboard, mouse, tablet, smartwatch, speaker, drone, car, groceries, food, medicine) *and* doesn't also match a real category or category group.

“That's not something CosmicMart carries today, but here's what I can help you find:” with buttons for Fashion / Home / Lifestyle. Clicking one re-seeds the query as “Show me something from {group}” and jumps straight to the budget question.

Bag / Checkout

- Empty state: “Your bag is empty. Ask Cosmo to find something for you ♦”
- Checkout button always fires: “This is a hackathon demo — checkout isn't wired to a real payment backend yet.”

Intent Parsing

Category, colour, and occasion are all extracted from the one free-text query. Season also checks the free text, defaulting to the real current calendar month's season if nothing's mentioned.

Keyword families (keywordMaps . ts):

- **Categories:** dress, jacket, sneakers, jeans, sweater, duvet, pillow, lamp, rug, wall-art, yoga-mat, water-bottle, backpack — each with a synonym list, plus a broader Fashion / Home / Lifestyle group fallback.

- **Colour families:** neutral, black, grey, blue, teal, green, yellow (includes mustard/gold/brass), orange, pink, red, purple, brown, multicolor — synonyms resolve on both the query side and the product's own colour tags, so “mustard” matches a product tagged “gold”.
- **Occasions:** wedding, party, work, gift, formal, everyday.

The Three Hard Gates (Recommendation Agent)

An item failing any one gate is dropped entirely — it never reaches scoring, regardless of how well it would otherwise fit:

- **Budget** — $\text{price} \leq \text{budget}$ (skipped entirely if the customer picked “No limit”)
- **Category** — exact match on `product.category` or `product.categoryGroup`
- **Colour** — the product's resolved colour-family set must contain the query's resolved colour family

Items that pass all three are scored:

```
score = budgetScore×0.35 + occasionScore×0.25 + ratingScore×0.25 + popularityScore×0.15
```

Budget score rewards closeness to (not just under) the stated budget. Occasion score is 100 on a tag match, 40 on a mismatch, 60 (neutral) if no occasion was given.

Orchestrator Fusion (final ranking)

Once all four agents return, scores are blended and the list is re-sorted:

```
finalScore = recScore×0.6 + trendScore×0.2 + friendInfluenceScore×0.2, minus 10 if out of stock
```

The list is sorted descending by `finalScore`. The first in-stock item (or the first item overall if none are in stock) is flagged as the Top pick.

Friend Agent Mechanics

Each session gets an in-memory room keyed by a generated code (CV-XXXXX, ambiguous characters like 0/O/1/I excluded). Votes are simulated deterministically — seeded by product id, so results are stable across re-renders rather than reshuffling — with an “up” bias that scales with the product's own popularity and rating, so more popular items appear more friend-loved.

No LLM calls happen anywhere in this flow — every response is deterministic keyword-matching and scripted branching, which is exactly what makes it reproducible from this spec.